ChatGPT said:

Red-Black Tree: Color Inversions — Notes

1. Introduction

A **Red-Black Tree** is a self-balancing Binary Search Tree (BST) in which each node is assigned either a red color or a black color. These colors help maintain balance during insertion and deletion operations.

A Red-Black Tree uses **recoloring (color changes) and rotations** to keep the tree approximately balanced and ensure efficient searching, insertion, and deletion.

2. Properties of a Red-Black Tree

A Red-Black Tree follows these five main properties:

1. **Node Color:** Every node is either red or black.
2. **Root Property:** The root node is always black.
3. **Leaf Property:** All NIL (null) leaves are black.
4. **Red Property:** A red node cannot have a red parent or a red child.
5. **Black Height Property:** Every path from a node to its descendant NIL leaves contains the same number of black nodes.

These properties ensure that the tree remains balanced.

3. What Is Color Inversion?

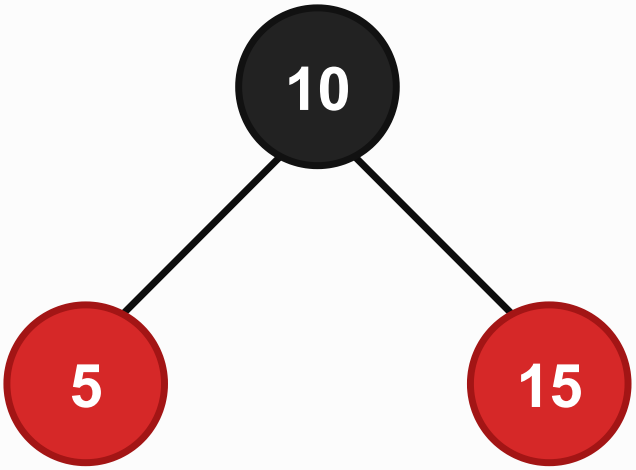
Color inversion, commonly called *recoloring*, is the process of changing the colors of nodes in a Red-Black Tree from red to black or from black to red to maintain the tree's properties.

Recoloring is generally performed when a newly inserted red node causes a violation of the Red-Black Tree rules.

For example, if a node and its parent are both red, the tree violates the Red Property. Recoloring may resolve this violation without requiring a rotation.

4. Example of Color Inversion

Suppose a Red-Black Tree has the following structure before recoloring:

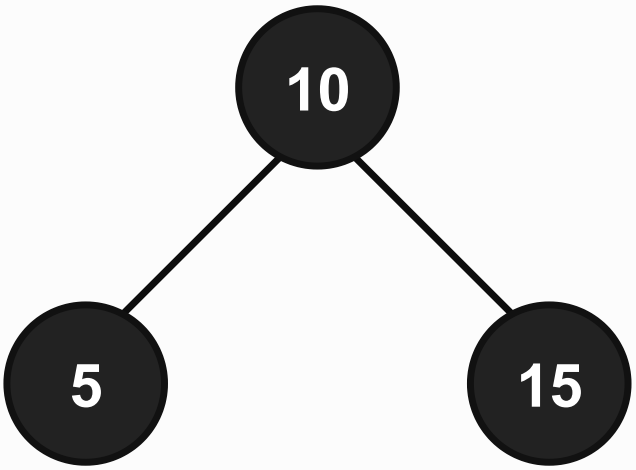


Root 10 is black; children 5 and 15 are red.

If a new red node is inserted under node 5, a red-red violation may occur.

When the parent and uncle are both red, recoloring can be applied.

After recoloring:



Nodes 5 and 15 change from red to black in this illustrative recoloring step.

Note: In a complete insertion example, the newly inserted node and the root may need additional color adjustments. The final tree must satisfy all Red-Black Tree properties.

5. When Is Recoloring Used?

Recoloring is commonly used during insertion when the parent of the newly inserted node is red.

- **Case 1: Parent is red and uncle is red.** Recolor the parent and uncle to black, and the grandparent to red. If the grandparent is the root, make it black.
- **Case 2: Parent is red and uncle is black, with a straight-line arrangement.** A rotation and recoloring are performed.
- **Case 3: Parent is red and uncle is black, with a zigzag arrangement.** First perform a rotation to convert it into a straight-line case, then rotate again and recolor.

During deletion, recoloring may also be used to resolve violations of the black-height property.

6. Difference Between Recoloring and Rotation

Recoloring (Color Inversion)	Rotation
Changes the color of nodes.	Changes the structural arrangement of nodes.
Does not change parent-child relationships.	Changes some parent-child relationships.
Helps resolve color-property violations.	Helps restore balance and resolve structural cases.

7. Advantages of Red-Black Trees

1. Maintains approximate balance automatically.
2. Provides $O(\log n)$ search, insertion, and deletion.
3. Avoids the excessive height of a skewed BST.
4. Is used in data structures such as ordered maps and sets in many programming libraries.

8. Conclusion

Color inversion, or recoloring, is an important operation in a Red-Black Tree. It changes node colors to maintain the tree's properties after insertion or deletion. Recoloring is often combined with rotations to preserve balance and ensure efficient operations.

Exam tip: When the parent and uncle are both red, recoloring is usually the first step. When the uncle is black, rotations and recoloring may be necessary.